



Séries temporais e sistema de recomendação

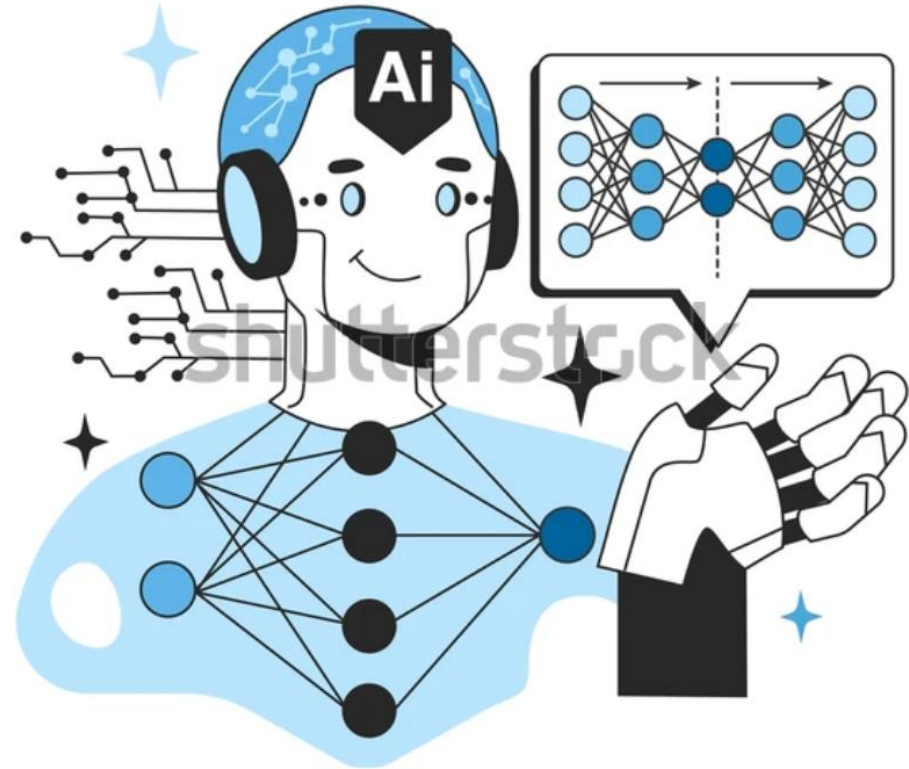
Fábio Santiago
fabio.santiago@facens.br



Introdução MLP regressora

Redes neurais artificiais aplicadas à regressão

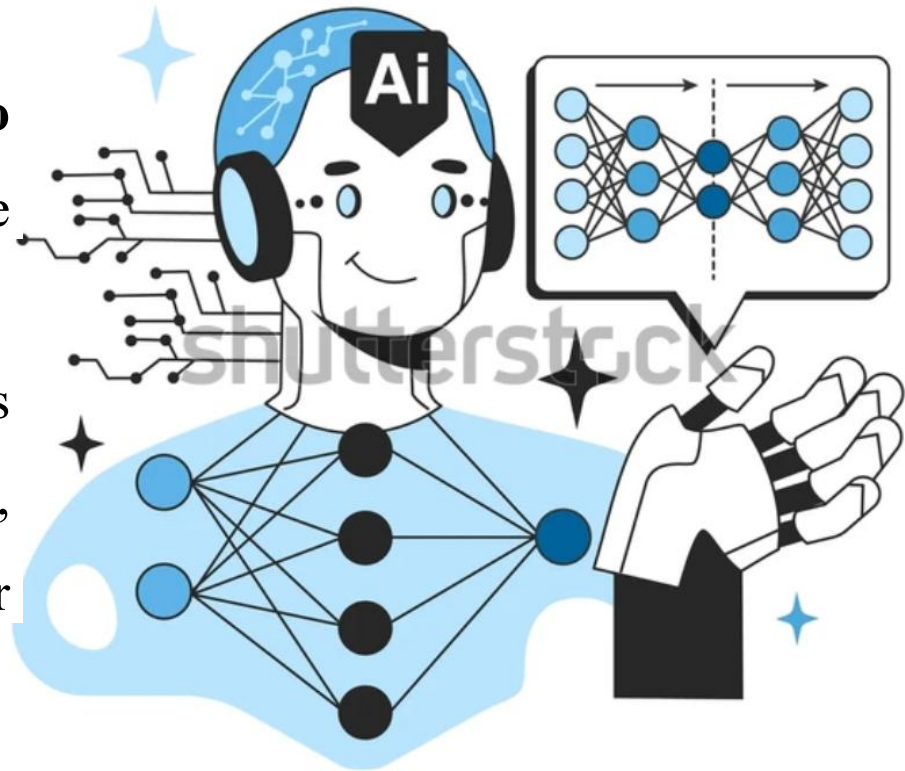
A aplicação de **Redes Neurais Artificiais (RNAs)** em problemas de **regressão** consiste em utilizar modelos de aprendizado profundo para prever **valores numéricos contínuos** a partir de dados de entrada. .



Redes neurais artificiais aplicadas à regressão

Objetivo da Regressão com RNAs:

1. Mapear **relações complexas e não lineares** entre variáveis de **entrada (X)** e uma saída **contínua (y)**.
2. Exemplos: Prever preços de imóveis (baseado em tamanho, localização, etc.), estimar consumo de energia, projetar vendas futuras.



Redes neurais artificiais aplicadas à regressão

Aplicações Práticas:

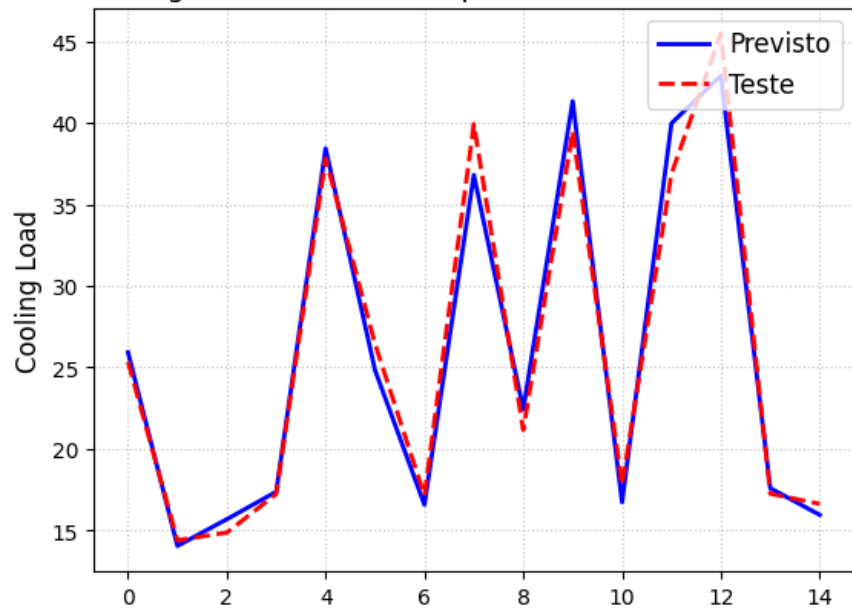
- **Finanças:** Previsão de ações, análise de risco.
- **Engenharia:** Estimativa de resistência de materiais, desgaste de equipamentos.
- **Saúde:** Predição de níveis de glicose no sangue, progressão de doenças.
- **Marketing:** Projeção de receita com base em investimento em campanhas.



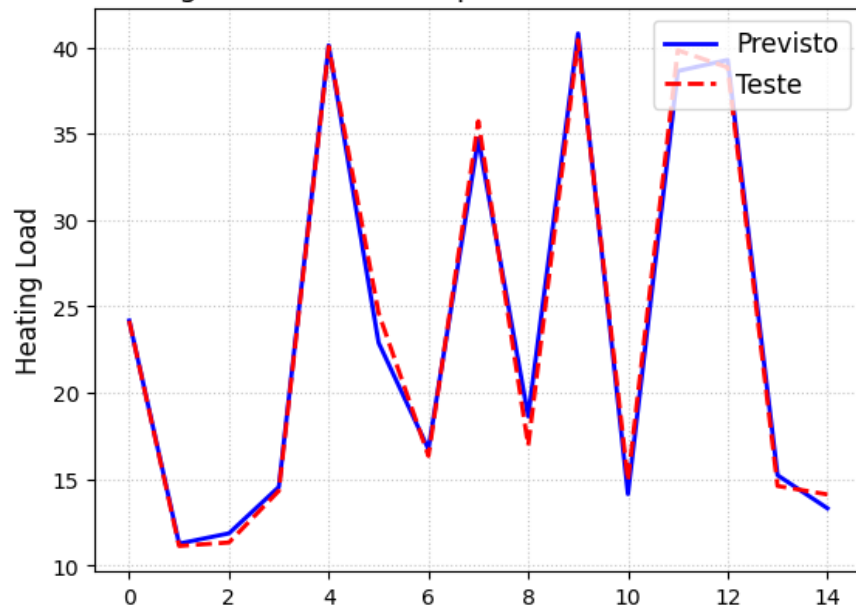
Redes neurais artificiais aplicadas à regressão

Exemplos

Cooling Load resultados previstos e os dados de teste

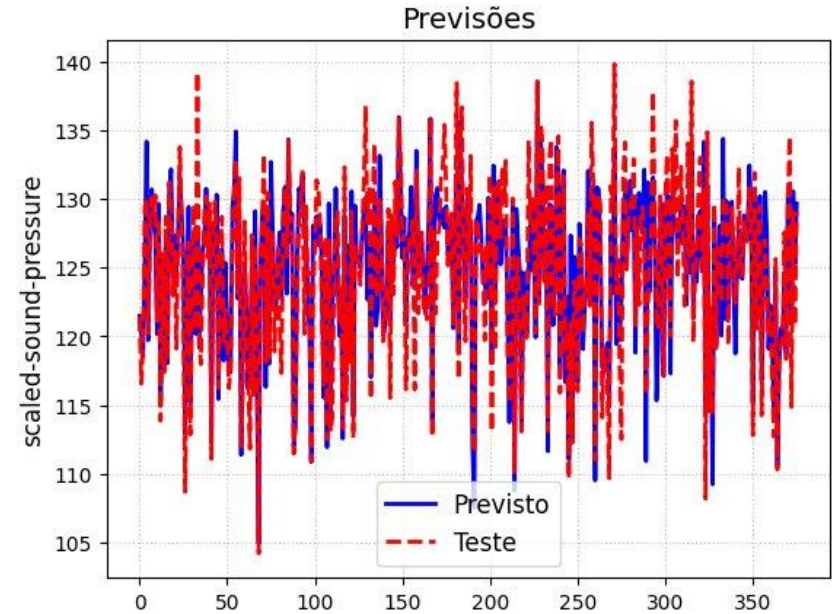
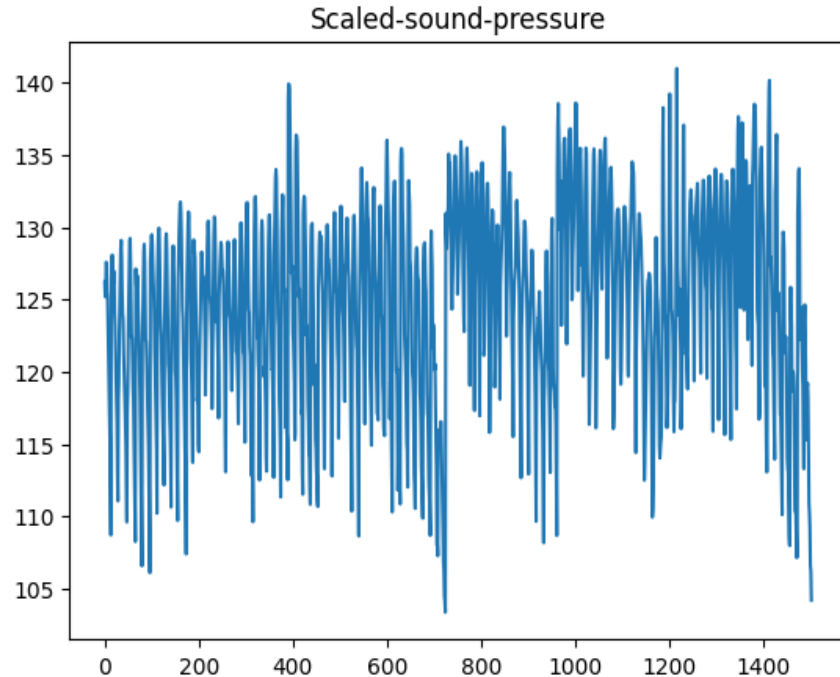


Heating Load resultados previstos e os dados de teste



Redes neurais artificiais aplicadas à regressão

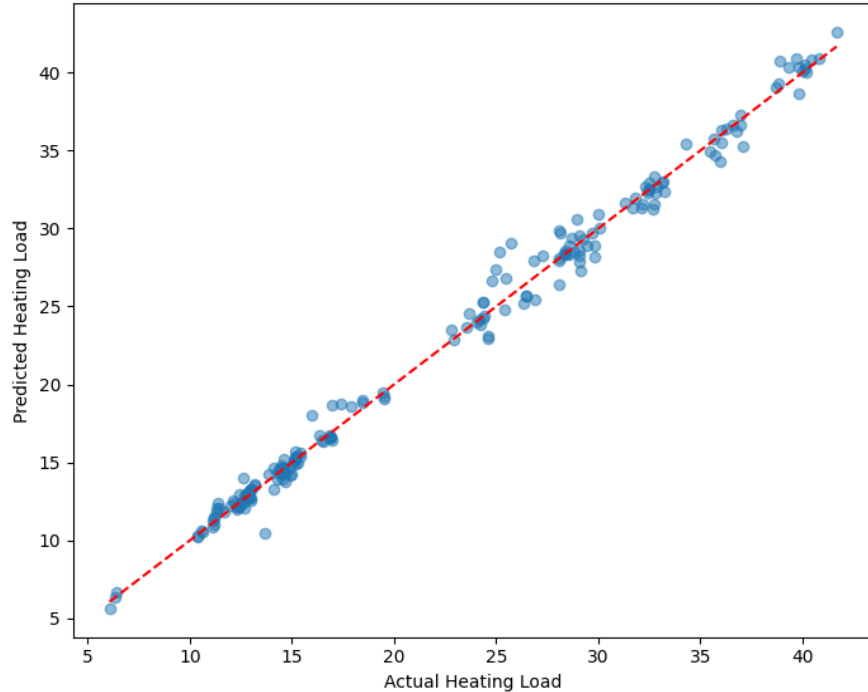
Exemplos



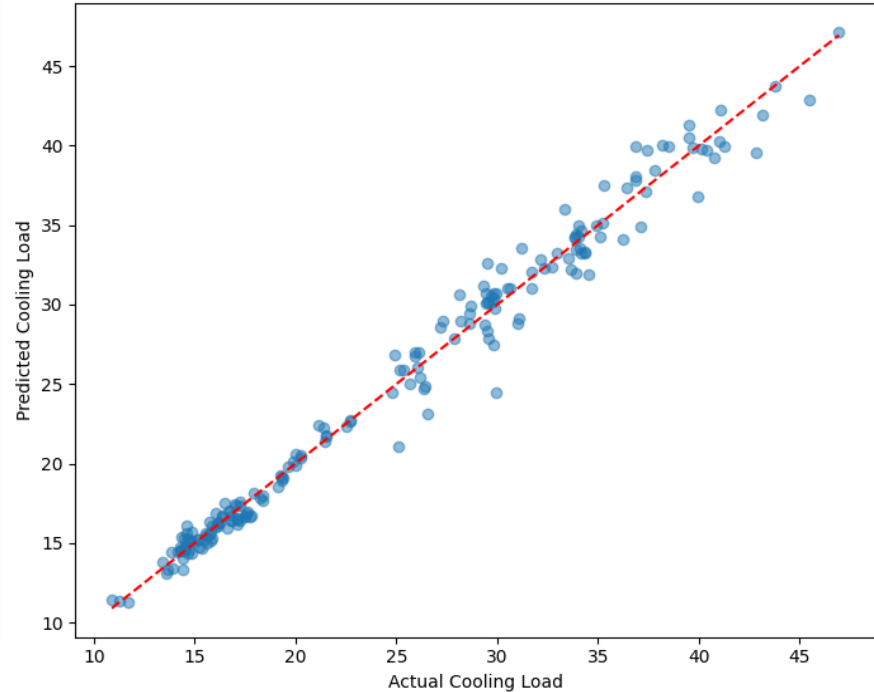
Redes neurais artificiais aplicadas à regressão

Exemplos

Heating Load Prediction



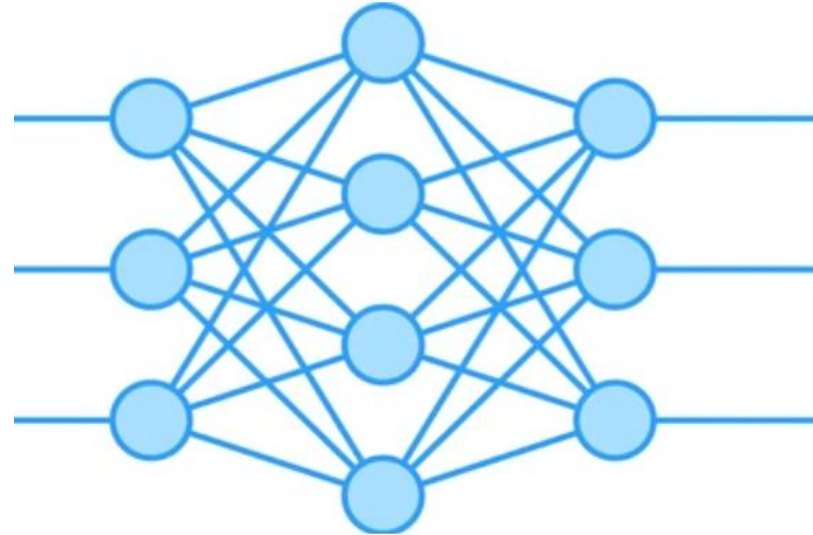
Cooling Load Prediction



Redes neurais artificiais aplicadas à regressão

Arquitetura típica para regressão:

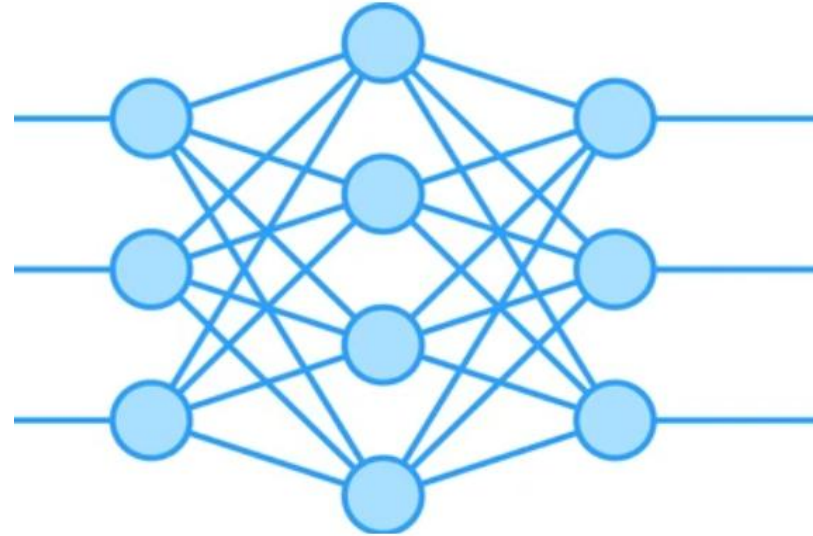
- **Camada de entrada:** No MLPRegressor do Scikit-Learn, o número de neurônios na camada de entrada não é especificado manualmente. Ele é determinado automaticamente com base no formato dos dados de entrada (X).



Redes neurais artificiais aplicadas à regressão

Arquitetura típica para regressão:

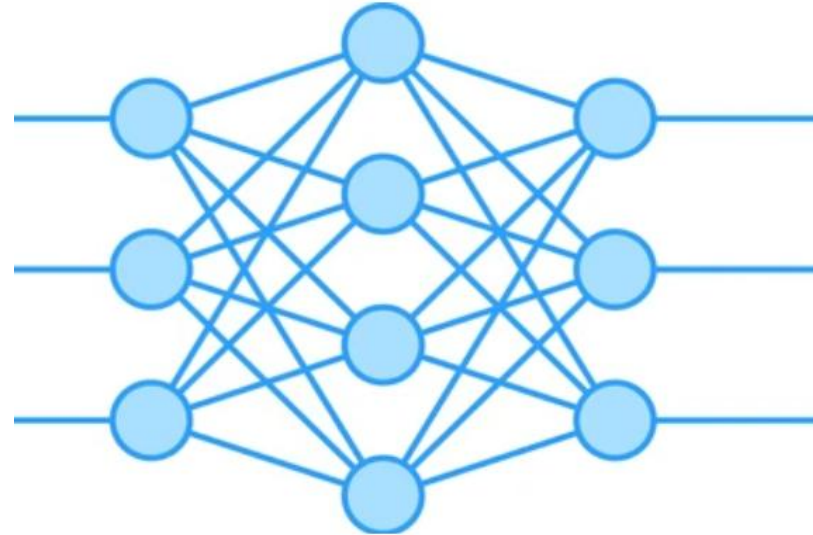
- A camada de entrada tem exatamente um neurônio para cada *feature* (variável) do seu dataset.
- Se seu conjunto de dados X tem shape $(n_amostras, n_features)$, a camada de entrada terá $n_features$ neurônios.



Redes neurais artificiais aplicadas à regressão

Arquitetura típica para regressão:

- **Camada oculta.** O número de neurônios na camada oculta de uma rede neural regressiva é crucial para o desempenho do model.

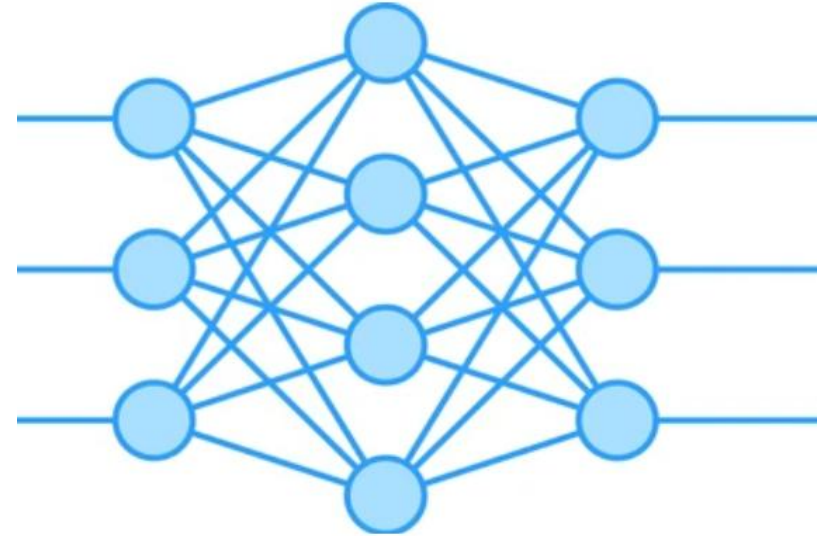


Redes neurais artificiais aplicadas à regressão

Diferentes fatores influenciam na escolha do número de neurônios a serem utilizados.

Tamanho do dataset

- **Dados pequenos** (<1k amostras): Menos neurônios (ex: 5-20) para evitar overfitting.
- **Dados grandes** (>10k amostras): Pode usar mais neurônios (ex: 50-200).



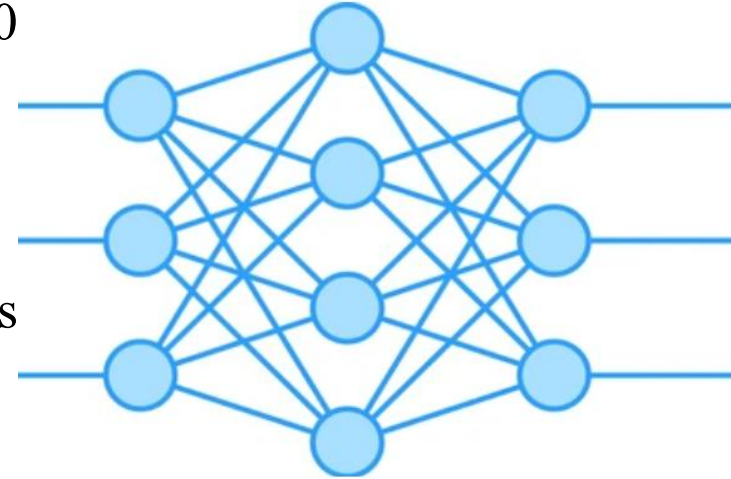
Redes neurais artificiais aplicadas à regressão

Complexidade do problema

- Relações lineares simples: 1-10 neurônios.
- Padrões não lineares complexos: 50-500 neurônios (ou mais em redes profundas).

Arquitetura da rede

- Redes rasas (1-2 camadas): Usar mais neurônios por camada.
- Redes profundas (3+ camadas): Diminuir gradualmente (ex: $256 \rightarrow 128 \rightarrow 64$).



Redes neurais artificiais aplicadas à regressão

Regras práticas e heurísticas a serem utilizadas para determinar o número de neurônios nas camadas ocultas

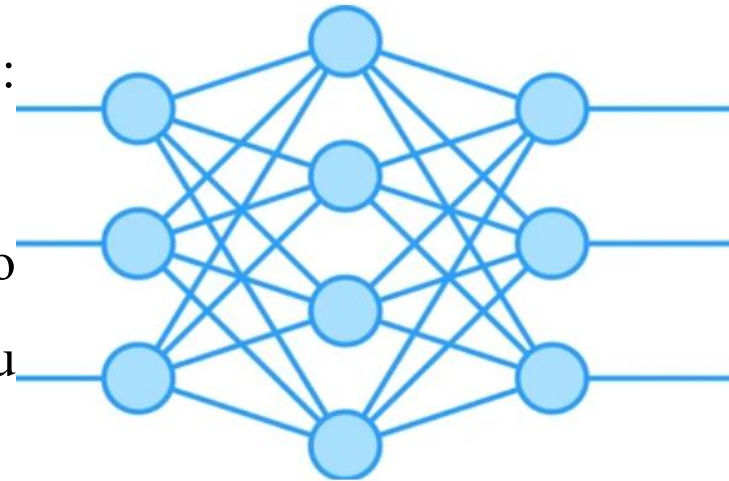
Método	Fórmula/Cálculo	Exemplo (Input=10, Output=1)
Média Aritmética	$\frac{n_{\text{input}} + n_{\text{output}}}{2}$	$\frac{10+1}{2} \approx 5$
Regra de Kolmogorov	$2 \times n_{\text{input}} + 1$	$2 \times 10 + 1 = 21$
Raiz Quadrada	$\sqrt{n_{\text{input}} \times n_{\text{output}}}$	$\sqrt{10 \times 1} \approx 3$
2/3 da Camada de Entrada	$\frac{2}{3} \times n_{\text{input}}$	$\frac{2}{3} \times 10 \approx 6$

Redes neurais artificiais aplicadas à regressão

Monitoramento

Acompanhe estes sinais para ajustar os neurônios:

- **Underfitting** (alta perda no treino/validação):
Aumente neurônios ou camadas.
- **Overfitting** (baixa perda no treino, alta no validação): Reduza neurônios ou adicione *Dropout*.



Redes neurais artificiais aplicadas à regressão

Função de Perda (Loss):

- **Erro Quadrático Médio (MSE):**

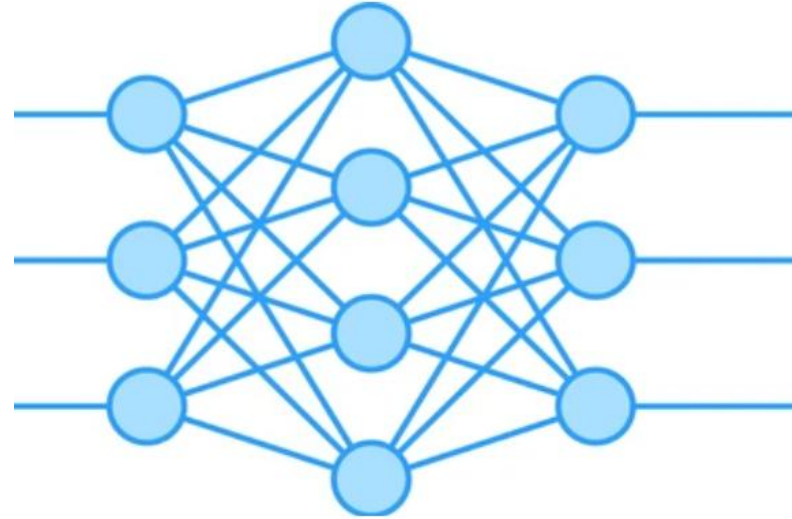
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Erro Absoluto Médio (MAE):**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Funções de Ativação nas Camadas Ocultas:**

ReLU, Tanh ou Sigmoid .



Redes neurais artificiais aplicadas à regressão

Desafios:

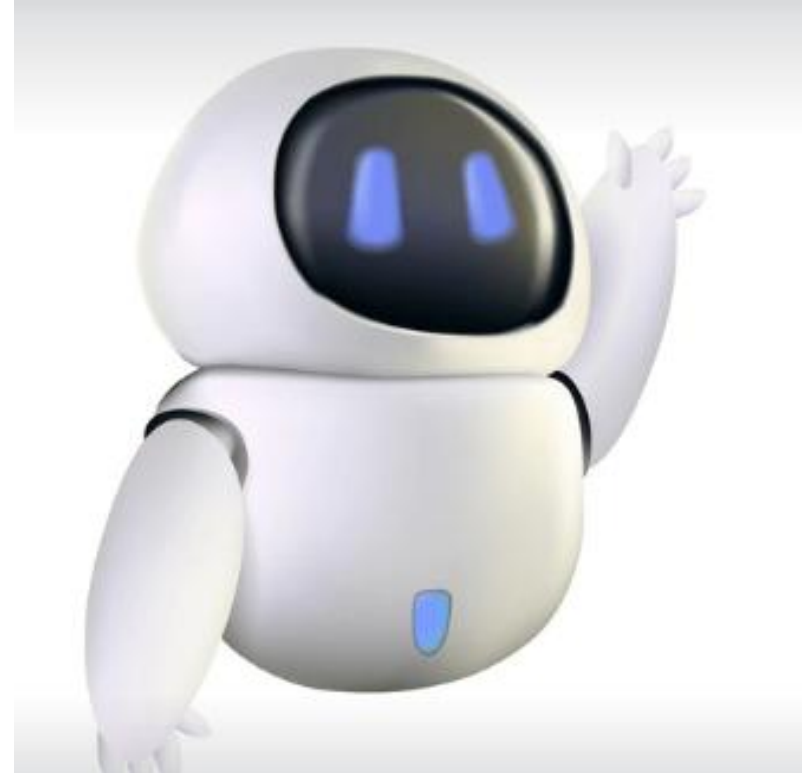
- **Escalonamento de dados:** Sensibilidade a variações de escala **normalizar** ou **padronizar entradas** é crucial.
- **Custo computacional:** Arquiteturas profundas demandam **alto poder de processamento**.



Redes neurais artificiais aplicadas à regressão

Vantagens sobre métodos clássicos:

- Capturam relações **não lineares** complexas.
- Lidam bem com **grandes volumes de dados** e **alta dimensionalidade**.
- Flexibilidade para integrar **dados estruturados e não estruturados** (imagens, texto).



MLPRegressor

O **MLPRegressor** (Multi-layer Perceptron Regressor) do Scikit-learn é uma implementação de redes neurais artificiais para problemas de regressão.



MLPRegressor

O **MLP (Multilayer Perceptron)** é composto por camadas de neurônios (perceptrons) para modelar relações complexas e não lineares entre as variáveis de entrada e a saída contínua.



MLPRegressor

Principais parâmetros:

- **Camadas Ocultas (hidden layers):** O MLPRegressor permite definir o número de camadas ocultas e o número de neurônios em cada camada. Por exemplo:
`hidden\_layer\_sizes=(100,)` significa uma camada oculta com 100 neurônios.
`hidden\_layer\_sizes=(50, 30)` significa duas camadas ocultas, a primeira com 50 neurônios e a segunda com 30.

MLPRegressor

Principais parâmetros:

- **Função de Perda (Loss Function):** O **MLPRegressor** minimiza o **Erro Quadrático Médio** (Mean Squared Error - MSE) por padrão. Essa função de perda é adequada para regressão. A perda é calculada como a média dos quadrados das diferenças entre os valores previstos e os reais.

MLPRegressor

Principais parâmetros:

Otimizador (**Solver**): O algoritmo usado para **otimizar os pesos da rede**. As opções são:

- **`lbfgs`**: Um otimizador quasi-Newton. **É mais estável para conjuntos pequenos.**
- **`sgd`**: Gradiente Descendente Estocástico (Stochastic Gradient Descent).
- **`adam`**: Uma extensão do SGD que adapta a taxa de aprendizado.

MLPRegressor

Principais parâmetros:

Taxa de Aprendizado (**Learning Rate**): Controla o tamanho do passo durante a atualização dos pesos. É crucial para a convergência. Quando se usa ``solver='sgd'`` ou ``solver='adam'``, pode-se definir:

- ``learning_rate_init``: A taxa de aprendizado inicial.
- ``learning_rate``: Pode ser ``constant`` (fixa), ``invscaling`` (decai com o tempo) ou ``adaptive`` (ajusta se a perda não melhora).

MLPRegressor

Principais parâmetros:

Iterações e Convergência

- **`max\_iter`**: Número máximo de iterações (épocas). O **treinamento para** quando a **convergência é alcançada** ou quando **`max\_iter` é atingido**.
- **`tol`**: Tolerância para a otimização. Se a melhoria da **perda for menor que `tol`** por um certo número de iterações (**`n\_iter\_no\_change`**), o treinamento para.

MLPRegressor

Principais parâmetros:

- **`batch\_size`**: Tamanho do lote para o SGD. Se **`batch\_size='auto'`**, usa-se `min(200, n_samples)`.
- **`shuffle`**: Se deve embaralhar os dados a cada iteração (para SGD e Adam).
- **`early\_stopping`**: Se deve usar uma parte dos dados como validação para parada antecipada. Parâmetro **`validation\_fraction`** define a fração de dados reservada para validação.

Pré-processamento

MinMaxScaler e StandardScaler no contexto de Redes Neurais

Ambos são técnicas de **pré-processamento de dados** essenciais para redes neurais. Sua função principal é transformar as features para uma escala adequada, melhorando a convergência e performance do modelo

Pré-processamento

MinMaxScaler : Normaliza os dados para um intervalo específico (geralmente $[0, 1]$ ou $[-1, 1]$).

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Características:

- Comprime os dados entre 0 e 1 por padrão
- Preserva a distribuição original
- Sensível a outliers

Pré-processamento

StandardScaler: Padroniza os dados para ter média 0 (zero) e desvio padrão 1(um).

$$X_{\text{std}} = \frac{X - \mu}{\sigma}$$

Características:

- Centraliza na média ($\mu = 0$)
- Escala pelo desvio padrão ($\sigma = 1$)
- Menos sensível a outliers que MinMax
- Pode gerar valores negativos

Aplicação MLPRegressor

Implementando



Aplicação MLPRegressor :Energy Efficiency



Aplicação MLPRegressor :Energy Efficiency

Dataset Energy Efficiency

Este estudo analisou a avaliação dos requisitos de carga de aquecimento e carga de resfriamento de edifícios (ou seja, eficiência energética) como uma função dos parâmetros de construção

Aplicação MLPRegressor :Energy Efficiency

Feature

- **X1 Relative Compactness** Continuous
- **X2 Surface Area** Continuous
- **X3 Wall Area** Continuous
- **X4 Roof Area** Continuous
- **X6 Orientation** Integer
- **X7 Glazing Area** Continuous
- **X8 Glazing Area Distribution**

Target

- **Y1 Heating Load** Continuous
- **Y2 Cooling Load** Continuous





Facens

AQUI TEM ENGENHARIA